

Medium Day is Sept 18. Sign up for free and discover something new! [Register now.](#)



Rust



Webgpu



Webassembly



3d Graphics



Open Source



Scenix v1.0.0: Building a Modular Rust-Native 3D Scene Toolkit for Native and WASM Apps



Aarambh Dev Hub

Follow

8 min read · May 27, 2026



13





I'm excited to share that **Scenix v1.0.0** is officially live.

This release is special for me because Scenix is no longer just an experiment, a roadmap, or a collection of ideas. With v1.0.0, it has reached its first stable milestone.

Scenix is a **modular Rust-native 3D scene toolkit** for building native and WASM-based 3D applications. It is designed for developers who want a clean Rust-first way to build scenes, work with cameras, create geometry, use materials and lights, load assets, perform raycasting, animate objects, and eventually render through `wgpu` and WebGPU.

Website / Demo: <https://aarambhdevhub.github.io/scenix/>

GitHub Release:

<https://github.com/AarambhDevHub/scenix/releases/tag/v1.0.0>

Why I Built Scenix

When working with 3D graphics, one thing becomes clear very quickly: everything grows fast.

At first, you only need a vector type and a transform.

Then you need a scene graph.

Then cameras.

Then mesh primitives.

Then materials.

Then lights.

Then textures.

Then asset loaders.

Then raycasting.

Then animation.

Then WASM support.

Then renderer integration.

And before you know it, a small 3D project becomes a big monolithic codebase where every feature depends on every other feature.

I wanted Scenix to avoid that from the beginning.

The main idea behind Scenix is simple:

Use only what you need. Build scenes the Rust way. Keep the system modular, fast, and developer-friendly.

That is why Scenix is not designed as one huge crate. It is built as a clean Cargo workspace with focused crates for each part of the 3D pipeline.

What Is Scenix?

Scenix is a **Rust-native 3D scene toolkit**.

It helps developers build and manage 3D scenes using Rust. It includes tools for:

- 3D math
- scene graphs
- transforms
- cameras
- mesh geometry
- materials

- lights
- textures
- loaders
- raycasting
- debug helpers
- post-processing
- animation through Animato
- WASM and browser support

The goal is not to replace a full game engine. Scenix is not trying to be Unity, Unreal, or Bevy.

Instead, Scenix focuses on being a strong **3D scene foundation**.

You can use it for game tools, creative coding, data visualization, 3D editors, browser demos, product viewers, rendering experiments, and Rust-native graphics projects.

Why Modular Crates Matter

One of the biggest design decisions in Scenix is the modular crate architecture.

Instead of forcing every developer to compile everything, Scenix lets you choose only the parts you need.

For example, if you only need math types, you can use:

```
scenix-math = "1"
```

If you only need a scene graph:

```
scenix-scene = "1"
```

If you want the full facade crate:

```
scenix = "1"
```

And if you want the full stack with optional features:

```
scenix = { version = "1", features = ["full"] }
```

This matters a lot in Rust.

Large graphics libraries can become heavy very quickly. Compile time, dependency size, feature flags, and optional GPU code all matter.

With Scenix, CPU-side crates like math, core, scene, mesh, camera, material, light, texture, helpers, and raycasting can be used without forcing every

project to depend on the full rendering stack.

That makes Scenix easier to adopt in smaller tools, tests, servers, editors, and experiments.

The Scenix Workspace

Scenix v1.0.0 is organized into focused crates:

scenix-math	→ vectors, matrices, quaternions, transforms, rays, bounds
scenix-core	→ IDs, traits, colors, shared errors
scenix-input	→ keyboard, pointer, touch, gamepad state
scenix-scene	→ scene graph, nodes, hierarchy, transforms
scenix-camera	→ perspective , orthographic, cube cameras, controllers
scenix-mesh	→ geometry, primitives, morph targets, instancing
scenix-material	→ PBR, physical, toon, unlit, wireframe, shader materials
scenix-light	→ ambient, directional, point, spot, hemisphere, area lights
scenix-texture	→ texture types, samplers, atlases, mipmaps
scenix-loader	→ GLTF, OBJ, STL, image , asset loading
scenix-renderer	→ wgpu rendering pipeline
scenix-post	→ bloom, SSAO, tone mapping, FXAA, TAA, outlines
scenix-raycaster	→ BVH raycasting and object picking
scenix-helpers	→ grid , axes, bounding box, light, camera helpers
scenix-animato	→ bridge between Animato and Scenix
scenix-wasm	→ WebGPU/WASM browser integration
scenix	→ facade crate for easier usage

Open in app ↗



Medium



Search



Write



1



Each crate has one responsibility.

A developer working on materials does not need to understand the WASM layer. A developer using only the scene graph does not need to compile `wgpu`.

A developer building browser demos can enable the WASM layer only when needed.

That is the kind of architecture I wanted from day one.

Powered by `wgpu` and WebGPU

Scenix is designed around `wgpu` for rendering.

That means the rendering layer can target modern GPU backends like Vulkan, Metal, DX12, and WebGPU from one Rust codebase.

This is important because Rust graphics should not be locked to only one platform.

The goal is to make Scenix useful for:

- native desktop apps
- browser-based WASM apps
- WebGPU demos
- creative tools
- visualization tools
- future editor-style applications

For browser support, Scenix includes a WASM layer through `scenix-wasm`.

The v1.0.0 website also includes a Leptos-based GitHub Pages demo, showing Scenix as a browser-ready project instead of only a native library.

Website / Demo: <https://aarambhdevhub.github.io/scenix/>

Animation Through Animato

Scenix is also connected with my animation project, **Animato**.

Animato handles how things move.

Scenix handles what things are, where they are, and how they appear in the scene.

The bridge crate, `scenix-animato`, connects animation tracks to Scenix objects.

That means Scenix can use Animato-driven values for:

- node transforms
- camera movement
- material colors
- opacity
- skeleton or bone transforms
- scene animation drivers

For v1.0.0, Scenix uses **Animato v1.4.0**.

This connection is important because I do not want Scenix to be just a static 3D library. I want it to become part of a bigger Rust creative ecosystem.

Scenix + Animato means:

3D scenes + animation logic Rust-native structure + motion system
rendering foundation + movement foundation

That combination is one of the most exciting parts of the project for me.

Raycasting, Helpers, and Developer Experience

One of the features I really wanted in Scenix was practical scene interaction.

That is where raycasting comes in.

With `scenix-raycaster`, developers can cast rays into a scene and detect which object is hit. This is useful for:

- object picking
- editor tools
- hover effects
- click selection
- debugging
- interactive demos
- 3D UI experiments

Scenix also includes debug helpers through `scenix-helpers`.

Helpers like grids, axes, bounding boxes, camera helpers, and light helpers are not just small utilities. They make development easier.

When building 3D scenes, visual debugging is important. You need to see where your objects are, how big they are, where the camera is pointing, and how your scene is structured.

Good helpers make a 3D library feel much more developer-friendly.

Why v1.0.0 Matters

The v1.0.0 release is not only about adding more features.

It is about stability.

For this release, the focus was:

- stable public API
- better documentation
- examples
- CI improvements
- WASM support
- website/demo
- modular features
- release notes
- cleaner developer experience

A stable release tells developers:

This project has a direction. This API is ready to be used. The foundation is now strong enough to build on.

Of course, Scenix will keep improving after 1.0.0. There are still many future ideas: editor tools, particles, terrain, XR, physics bridges, advanced rendering, and more.

But v1.0.0 is the foundation.

Install Scenix

The simplest way to start:

```
[dependencies]
scenix = "1"
```

For a full-feature setup:

```
[dependencies]
scenix = { version = "1", features = ["full"] }
```

Or install only the crates you need:

```
scenix-math = "1"
scenix-scene = "1"
```

```
scenix-camera = "1"  
scenix-raycaster = "1"
```

This is the part I personally like most: Scenix does not force one workflow.

You can use it as a full toolkit, or you can use only one layer.

A Small Example

Here is a simple idea of what using Scenix looks like:

```
use scenix::*;  
  
fn main() {  
    let mut scene = SceneGraph::new();  
    let root = scene.add(SceneNode::new("Root"));  
    let cube = scene  
        .add_child(  
            root,  
            SceneNode::new("Cube")  
                .transform(Transform::from_translation(Vec3::new(0.0, 1.0, 0.0))  
        )  
        .expect("cube should be added");  
    scene.update_world_transforms();  
    println!("Created scene with cube node: {:?}", cube);  
}
```

This example does not even need GPU rendering.

That is the point.

You can build, test, inspect, and manage scenes before touching the rendering layer.

What Scenix Can Be Used For

Scenix can be useful for many types of projects:

Game Tools

Not every Rust game tool needs a full game engine. Sometimes you need a scene graph, camera, mesh tools, asset loading, or preview rendering.

Creative Coding

Rust is powerful for creative systems. Scenix can become a base for procedural scenes, animations, visuals, and generative art.

3D Data Visualization

Charts are not always enough. Some data needs space, depth, interaction, and movement.

WebGPU Experiments

With WASM and WebGPU support, Scenix can be used for browser-based 3D experiments.

Custom Editors

Scene graph, helpers, raycasting, cameras, and object picking are the foundation of editor tools.

Product Viewers

A lightweight 3D model viewer with GLTF loading, orbit controls, materials, and post-processing is a practical use case.

What I Learned While Building It

Building Scenix taught me that architecture matters a lot.

It is easy to add features.

It is harder to keep features clean.

A 3D library can become messy quickly if every module depends on every other module. That is why crate boundaries were important.

I also learned that documentation is not optional.

For open-source projects, the code is only one part. Developers need:

- installation instructions
- examples
- architecture docs
- feature explanations
- migration notes
- troubleshooting
- release notes
- API docs

If people cannot understand how to use a project, they will not use it, no matter how good the code is.

That is why Scenix v1.0.0 also focuses on documentation and developer experience.

The Road Ahead

Scenix v1.0.0 is stable, but it is not the end.

It is the beginning of the next phase.

Some future ideas include:

- better visual demos
- more renderer improvements
- editor-style tools
- particle systems
- terrain rendering
- physics bridge
- XR support
- more examples
- more real-world model viewers
- deeper Animato integration

I want Scenix to grow slowly, cleanly, and practically.

Not by adding random features, but by building things developers actually need.

Final Thoughts

Scenix started as an idea:

What if Rust had a modular 3D scene toolkit that could work across native and WASM apps?

With v1.0.0, that idea now has a stable foundation.

It has a workspace structure.

It has crates.

It has documentation.

It has examples.

It has a website.

It has a release.

And most importantly, it has a direction.

Scenix is still growing, and I am still learning while building it. But this release feels like a real milestone.

Scenix v1.0.0 is live.

The stable journey starts now.

Website / Demo: <https://aarambhdevhub.github.io/scenix/>

GitHub Release:

<https://github.com/AarambhDevHub/scenix/releases/tag/v1.0.0>

GitHub Repository: <https://github.com/AarambhDevHub/scenix>

#RustLang #RustProgramming #WebGPU #WASM #WebAssembly #WGPU
#GraphicsProgramming #3DGraphics #GameDevelopment #OpenSource
#CratesIO #DeveloperTools #CreativeCoding #Rendering #SceneGraph #PBR
#Raycasting #GLTF #Leptos #AarambhDevHub #SystemsProgramming

Rust

+

Webgpu

+

Webassembly

+

3d Graphics

+

Open Source

+



Written by Aarambh Dev Hub

269 followers · 1 following

Follow

Rust developer sharing coding tutorials, backend tips, and insights. Follow for deep dives into Rust, programming challenges, and modern development practices.

